

COLOUR KEY: Keyboard Slash cmd Workflow Skill/agent CLI/flag Config/env ★ New in 2026 Removed/deprecated

Getting the Most Out of Claude Code

A practical guide to operating Claude Code like an expert — verified April 24 2026, v2.1.118

Claude Code is an **agentic coding assistant** — it reads your repo, runs commands, edits files, and manages git workflows on its own, within the permission boundaries you set. The gap between casual and expert use is narrow but compounds quickly: the practitioners who get the most out of it **plan before they build**, give Claude **durable project memory**, isolate risky work in **worktrees**, and offload repeated thinking to **skills, plugins, and subagents**.

How to read this sheet. Entries shown in their section colour are stable features. **Pink entries marked with a ★** are new in 2026 (v2.1.84 or later) and may require a current install or specific plan tier. **Grey struck-through entries** have been removed or deprecated and should be avoided in scripts. Pages are organised by how you actually work: **Page 1** is the complete slash-command reference, **Page 2** covers keyboard shortcuts and vim mode, **Page 3** documents CLI flags, environment variables, permission modes, and hook events, and **Page 4** catalogues the official plugin marketplace.

The short path from competent to expert. For anything touching more than a couple of files, start in plan mode (**Shift+Tab** cycles permission modes, or type **/plan**) and let Claude propose a structured approach before editing. Put essential project context in a **CLAUDE.md** at your repo root — it survives compaction and grounds every turn. Codify repeated workflows as skills in **.claude/skills/**. Isolate parallel or risky work with **--worktree**. Ask side questions with **/btw** to keep the main conversation clean. Raise effort with **/effort high** or **xhigh** for hard problems on Opus 4.7. And install an **LSP plugin** for your language so Claude sees type errors in real time as it edits.

/ SESSION & CONTEXT

/clear Clear history — aliases: **/reset**, **/new**
/compact [instr] Compact conv with optional focus
/context Visualise context usage as grid
/copy [N] Copy last (or Nth) response
/diff Interactive diff — uncommitted + per-turn
/export [file] Export conversation as plain text
/rewind Rewind conv/code (alias: **/checkpoint**)
/branch [name] Branch conversation (alias: **/fork**)
/resume [session] Resume by ID/name (alias: **/continue**)
/rename [name] Rename session; shows on prompt bar
/recap Show session context recap
/btw <question> Side question — no context cost

Model & Effort

/model [model] Pick/change model; **↔** adjusts effort
/effort [level] low/medium/high/xhigh★/max/auto
/fast [on|off] Toggle fast mode

Display & Config

/config Open settings (alias: **/settings**)
/theme Change theme — now supports custom ★
/color [color] Set prompt-bar color for session
/tui [fullscreen] Flicker-free fullscreen rendering
/focus Toggle focus view
/statusline Configure shell prompt status line
/keybindings Open/create keybindings config
/terminal-setup Configure Shift+Enter shortcut
/permissions Manage allow/ask/deny rules
/status Settings — version, model, network

Input Prefixes

/ Slash command or skill
! Direct bash — output enters context
@ File mention + autocomplete

/ TOOLS & INTEGRATION

/init Initialize project CLAUDE.md
/memory Edit CLAUDE.md; manage auto-memory
/mcp Manage MCP server connections
/hooks View hook configurations
/skills List skills — t sorts by tokens
/agents Manage agent configurations
/plugin Plugin manager (alias: **/plugins**)
/chrome Configure Claude in Chrome
/reload-plugins Hot-reload plugins
/ide Manage IDE integration status
/add-dir <path> Add working directory
/install-github-app Set up Claude GitHub Actions
/install-slack-app Install Claude Slack app (OAuth)
/web-setup Connect GitHub to Claude Code on web

Workflows & Agentic

/plan [desc] Enter plan mode with optional task
/batch <instr> 5–30 worktree agents [Skill]
/simplify [focus] 3-agent parallel code review [Skill]
/debug [desc] Debug logging + analysis [Skill]
/loop [interval] Recurring task; self-paces [Skill]
/schedule [desc] Create/run routines
/ultraplan <prompt> Cloud plan editor → web UI
/ultrareview [PR#] Parallel multi-agent PR review
/autofix-pr [prompt] Web agent auto-fix PR CI/comments
/security-review Scan branch diff for vulnerabilities
/team-onboarding Replayable guide from 30-day history
/less-permission-prompts Propose allowlist from transcripts
/claude-api Load API reference [Skill]

Remote & Sessions

/remote-control Enable remote control (alias: **/rc**)
/teleport /tp Pull web session into local terminal
/remote-env Configure default remote environment
/voice Toggle push-to-talk dictation
/desktop /app Continue in Desktop app (mac/Win)
/mobile /ios QR code for Claude mobile app
/sandbox Toggle sandbox mode
/tasks List background tasks (alias: **/bashes**)

/ UTILITY

/usage Plan limits + rate status
/stats Opens /usage → Stats tab
/cost Opens /usage → Cost tab
/extra-usage Keep working past rate limits
/insights Report on patterns + friction
/doctor Diagnose install; f = auto-fix
/powerup Interactive tutorials + demos
/release-notes Changelog — version picker
/feedback /bug Submit feedback
/help Show help and commands
/exit /quit Exit the CLI
/login /logout Sign in / out
/mcp elicitation MCP server structured-input dialog

Removed / Deprecated

~~**/pr-comments**~~ Removed v2.1.91 — ask Claude directly
~~**/vim**~~ Removed v2.1.92 — use **/config** → Editor
~~**/review**~~ Deprecated — install code-review plugin
~~**/tag**~~ Removed in v2.1.92

MCP Prompts as Commands

MCP servers expose prompts as commands:
/mcp__<server>__<prompt>
They appear automatically in the **/** menu.

★ QUICK TIPS

- Type **/** to see every command filtered live
- Skills (bundled or user-made) share the same UI as built-in commands
- Some commands are platform-dependent
(/desktop mac+win only, /upgrade on Pro/Max only)
- Plugin commands are namespaced:
/<plugin-name><command>

Official Effort Levels

low ■ medium ■ high ■ **xhigh ★** ■ max
xhigh — new in v2.1.111, default for Opus 4.7

COLOUR KEY:

Keyboard

Slash cmd

Workflow

Skill/agent

CLI/flag

Config/env

★ New in 2026

Removed/deprecated

■ GENERAL CONTROLS

Ctrl

C

Cancel input / generation

Ctrl

D

Exit session (EOF)

Ctrl

L

Clear prompt + force redraw

Ctrl

O

Toggle transcript viewer

Ctrl

R

Reverse search command history

Ctrl

G

Open prompt in editor

Ctrl

X

Ctrl

E

Open in editor (readline)

Ctrl

B

Background task (tmux: twice)

Ctrl

T

Toggle task list

Ctrl

V

Paste image → [Image #N] chip

Ctrl

X

Ctrl

K

Kill bg agents (2× in 3s)

Esc

Esc

Rewind / summarize

↑

↓

Cursor / history navigation

←

→

Cycle dialog tabs

Ctrl

P

Up — history nav

Ctrl

N

Down — history nav

Text Editing

Ctrl

A

Start of current line

Ctrl

E

End of current line

Ctrl

K

Delete to end of line

Ctrl

U

Delete to start of line

Ctrl

W

Delete previous word

Ctrl

Y

Paste deleted text (yank)

Alt

Y

Cycle paste history ■

Alt

B

Back one word ■

Alt

F

Forward one word ■

■ Requires Option-as-Meta on macOS (see box →)

■ MODE & NAVIGATION

Permission & Model Modes

Shift

Tab

Cycle permission modes

Alt

M

Cycle permission modes (alt)

Option

P

Switch model (mac)

Alt

P

Switch model (Win/Linux)

Option

T

Toggle extended thinking

Alt

T

Toggle thinking (Win/Linux)

Option

O

Toggle fast mode (mac) ⬆

Alt

O

Toggle fast mode (Win/Linux)

Transcript Viewer (Ctrl+O)

Ctrl

E

Toggle show all content

I

Search transcript

n

Next match

N

Previous match

I

Dump to scrollbar

v

Open in \$VISUAL/\$EDITOR

q

Exit transcript

Esc

Exit transcript

Voice Dictation

Space

Hold or tap to dictate

/voice tap = tap-to-toggle mode

20 langs: EN ES FR DE CZ PL KO JA ZH...

Session Picker (in /resume)

↑

↓

Navigate sessions

P

Preview

R

Rename

I

Search

A

All projects

B

Current branch

Prompt Suggestions

Tab

Accept inline suggestion

→

Accept inline suggestion

Enter

Accept + submit

■ VIM EDITOR MODE

Enable: /config → Editor mode

Mode Switching

Esc

NORMAL mode

i

Insert before cursor

a

Insert after cursor

I

Insert at line start

A

Insert at line end

o

O

Open line below / above

v

Visual (character) ★

V

Visual line ★

Navigation (NORMAL)

h

j

k

l

Move left/down/up/right

w

e

b

Word next/end/previous

O

S

A

Line start / end / non-blank

gg

G

Top / bottom of input

f

c

F

c

Jump to next/prev char

t

c

T

c

Jump before/after char

Editing (NORMAL)

x

dd

D

Del char / line / to end

cc

C

Change line / to end

dw

cw

yw

Word: del / change / yank

yy

Y

Yank line

p

P

Paste after / before

u

.

Undo / repeat

>>

<<

Indent / dedent

■ macOS OPTION-AS-META SETUP

Alt+B, Alt+F, Alt+Y, Alt+M, Alt+P, Alt+T

need Option configured as Meta:

• iTerm2: Profiles → Keys → Left Option

= "Esc+"

• Terminal.app: Profiles → Keyboard →

check "Use Option as Meta Key"

• VS Code: set

"terminal.integrated.macOptionsMeta":true

Beyond the Cheat Sheet — Tips to Save Tokens and Work Smarter

Four habits that separate heavy spenders from efficient operators

1. Know where your tokens actually go.

Every turn sends the full conversation, every active tool schema, every loaded **CLAUDE.md** , and the system prompt. Run **/context** regularly to see the breakdown as a grid — you will often find memory bloat, oversized tool results, or a stale conversation dominating the budget. Check **/usage** (formerly /cost) after expensive operations to see what you actually paid for. The fastest wins come from finding the single biggest offender on that screen rather than micro-optimising everywhere.

2. Plan before you spend.

Exploratory back-and-forth is the most common source of waste. Enter plan mode (**Shift+Tab** to cycle or **/plan**) for anything non-trivial — Claude researches and proposes an approach in one pass, and you approve before any editing begins. Use **/compact focus on <topic>** proactively rather than waiting for auto-compact at 95% — focused compaction keeps the right context alive. Put durable facts in **CLAUDE.md** so they survive compaction instead of being re-explained every session.

QUICK WINS

> --bare for scripted one-shots skips plugins/hooks/skills (faster startup)

> --max-budget-usd N caps any headless run before it runs away

> Prefer Grep/Glob over Read for discovery — full reads cost a lot

> Collapse bulky MCP results with anthropic/maxResultSizeChars

3. Match the model and effort to the task.

Opus 4.7 at **xhigh** effort is extraordinary for complex agentic coding, but routine edits, formatting, and lookups run fine on Sonnet 4.6 at **medium** . **/effort low** is often enough for "rename this variable everywhere" work. Subscription users get a 1-hour prompt cache TTL automatically; API-key users can opt in with **ENABLE_PROMPT_CACHING_1H=1** . Avoid switching models mid-conversation — the next turn re-reads the full history uncached.

4. Delegate — do not clutter.

Spawn the **Explore** subagent (read-only, Haiku, cheaper) for codebase navigation instead of doing it in the main context. Use **/btw** for side questions — it reuses the parent prompt cache and never enters history. Prefer **@path/to/file.md** over pasting contents. For truly parallel work, launch separate **--worktree** sessions with focused contexts rather than one session juggling three topics. And if you catch yourself re-explaining the same workflow a third time, codify it as a **skill** in **.claude/skills/** .

Press ? in Claude Code for shortcuts specific to your terminal · Source: code.claude.com/docs/en/interactive-mode

Page 2 / 5 — Keyboard & Interactive

COLOUR KEY: Keyboard Slash cmd Workflow Skill/agent CLI/flag Config/env ★ New in 2026 Removed/deprecated

■ CLI COMMANDS

claude Start interactive session
claude "query" Session with initial prompt
claude -p "query" Headless / print mode
claude -c Continue most recent conv
claude -r "name" Resume by name or ID
claude update Update to latest version
claude install [ver] Install specific / stable
claude auth login Sign in (--console for API)
claude auth logout Log out of account
claude auth status Auth status (--text = human)
claude agents List configured subagents
claude setup-token Long-lived token for CI
claude remote-control Start remote-control server
claude mcp Configure MCP servers
claude plugin Manage plugins
claude auto-mode defaults Print classifier rules

Key CLI Flags

--model sonnet / opus / full name
--permission-mode default/acceptEdits/plan/auto★/bypass
--effort low/med/high/xhigh★/max
--output-format text / json / stream-json
--max-turns N Limit agentic turns
--max-budget-usd N Cost cap per session
--worktree / -w Isolated git worktree
--tmux Create tmux pane for worktree
--bare Faster startup — skip plugins
--agent <name> Specify agent for session
--agents {JSON} Define agents dynamically
--from-pr N Resume PR-linked sessions
--fork-session New ID when resuming
--name / -n "x" Set session display name
--remote "task" Create web session
--remote-control / --rc Enable RC for session
--teleport Pull web → local terminal
--mcp-config <file> Load MCP from JSON
--strict-mcp-config Only use --mcp-config
--settings <file> Load extra settings
--plugin-dir <path> Load plugins from dir
--json-schema {s} Validated JSON output
--system-prompt Replace system prompt
--append-system-prompt Append to system prompt
--allowedTools Tools auto-allowed
--disallowedTools Tools removed
--tools Restrict built-in tools
--dangerously-skip-permissions Bypass ALL checks
--debug [cats] Debug w/ category filter
--verbose Full turn-by-turn logs
--version / -v Show version

■ CONFIG & ENV VARIABLES

Config Files & Scopes

~/.claude/settings.json User (global)
.claude/settings.json Project (shared via git)
.claude/settings.local.json Local (gitignored)
managed-settings.json Admin-deployed (read-only)
~/.claude.json OAuth & MCP state
~/.claude/themes/ Custom themes ★
~/.claude/keybindings.json Custom keybindings
~/.claude/CLAUDE.md Global personal memory
CLAUDE.md (project root) Project memory (survives compact)
.claude/rules/*.md Rule fragments loaded by path

Authentication

ANTHROPIC_API_KEY API key (overrides subscription)
CLAUDE_CODE_OAUTH_TOKEN Long-lived OAuth token for CI
ANTHROPIC_AUTH_TOKEN Custom Authorization header
ANTHROPIC_BASE_URL Proxy / gateway endpoint
ANTHROPIC_CUSTOM_HEADERS Custom request headers

Providers

CLAUDE_CODE_USE_BEDROCK Use AWS Bedrock
CLAUDE_CODE_USE_VERTEX Use Google Vertex AI
CLAUDE_CODE_USE_FOUNDRY Use Microsoft Foundry
CLAUDE_CODE_USE_MANTLE Use Bedrock Mantle endpoint
AWS_BEARER_TOKEN_BEDROCK Bedrock API key
HTTP_PROXY / HTTPS_PROXY Proxy configuration
NO_PROXY Bypass proxy for domains/IPs

Behaviour & Features

DISABLE_AUTOUPDATER Disable auto-updates
DISABLE_UPDATES Block ALL updates incl. manual
CLAUDE_CODE_NO_FLICKER Enable fullscreen rendering
CLAUDE_CODE_DISABLE_MOUSE Disable mouse tracking (fullscr)
CLAUDE_CODE_EFFORT_LEVEL Set effort level globally
CLAUDE_CODE_SIMPLE Set by --bare (minimal mode)
CLAUDE_CODE_DISABLE_BACKGROUND_TASKS Disable bg tasks + Ctrl+B
CLAUDE_CODE_DISABLE_CRON Disable /loop + scheduled tasks
CLAUDE_CODE_DISABLE_1M_CONTEXT Block 1M context models
CLAUDE_CODE_DISABLE_TERMINAL_TITLE Stop setting terminal title
CLAUDE_CODE_DISABLE_THINKING Force-disable extended thinking
MAX_THINKING_TOKENS Thinking budget (0 = disable)
MAX_MCP_OUTPUT_TOKENS MCP tool response cap (25000)
CLAUDE_CODE_MAX_OUTPUT_TOKENS Output token limit
BASH_DEFAULT_TIMEOUT_MS Bash cmd timeout (120000)
BASH_MAX_TIMEOUT_MS Max bash timeout (600000)
API_TIMEOUT_MS API timeout (600000 = 10min)
CLAUDE_AUTOCOMPACT_PCT_OVERRIDE Compact threshold % (def 95)
ENABLE_PROMPT_CACHING_1H Request 1-hour cache TTL
CLAUDE_CONFIG_DIR Override config dir (~/.claude)
CLAUDE_CODE_SUBPROCESS_ENV_SCRUB Scrub secrets from subprocesses
CLAUDE_ENV_FILE Sourced before each Bash cmd
CLAUDE_CODE_TMPDIR Override temp directory

■ PERMISSION MODES & HOOKS

Permission Modes (Shift+Tab cycles)

default Prompt on risky actions
acceptEdits Auto-allow edits; prompt on Bash
plan Read-only research / planning
auto Classifier allows safe / denies risky
dontAsk Deny silently, no prompt
bypassPermissions Skip ALL checks (dangerous)

Permission Rule Syntax

Bash(git *) — any git subcommand
Bash(git log *) — specific subcmd
Edit(src/**/*.ts) — glob file writes
Read(/-/*.env) — read-only paths
mcp__<server> — MCP tools by server
mcp__<server>__<tool> — specific tool

Hook Events (lifecycle)

SessionStart Session begins or resumes
SessionEnd Session terminates
UserPromptSubmit Before Claude processes prompt
UserPromptExpansion Before user cmd → prompt
PreToolUse Before tool call (can block)
PostToolUse After tool call succeeds
PostToolUseFailure After tool call fails
PostToolBatch After parallel batch resolves
PermissionRequest Permission dialog appears
PermissionDenied Auto-mode denial (can retry)
SubagentStart Subagent spawned
SubagentStop Subagent finishes
TaskCreated Via TaskCreate
TaskCompleted Task marked complete
Stop Claude finishes responding
StopFailure Turn ended on API error
PreCompact Before context compaction
PostCompact After compaction completes
Elicitation MCP requests input mid-call
ElicitationResult After user responds to MCP
Notification Claude sends notification
CwdChanged Working dir changes
FileChanged Watched file changes on disk
ConfigChange Config file changes
InstructionsLoaded CLAUDE.md/rules loaded
WorktreeCreate/Remove Worktree lifecycle
TeammateIdle Agent-teams teammate idle

Hook Types

command Run shell command
http POST event JSON to URL
mcp_tool Call MCP server tool
prompt Evaluate with LLM
agent Run agentic verifier

COLOUR KEY: Keyboard Slash cmd Workflow Skill/agent CLI/flag Config/env ★ New in 2026 Removed/deprecated

CODE INTELLIGENCE (LSP)

Real-time diagnostics + go-to-def after each edit.
Install binary FIRST, then plugin.

Available LSP Plugins

- typescript-lsp** TS/JavaScript
Binary: npm i -g typescript-language-server typescript
- pyright-lsp** Python
Binary: pip install pyright
- rust-analyzer-lsp** Rust
Binary: rustup component add rust-analyzer
- gopls-lsp** Go
Binary: go install golang.org/x/tools/gopls@latest
- clangd-lsp** C / C++
Binary: brew install llvm
- csharp-lsp** C#
Binary: dotnet tool install -g csharp-ls
- jdts-lsp** Java
Binary: brew install jdts
- kotlin-lsp** Kotlin
Binary: kotlin-language-server release
- swift-lsp** Swift
Binary: Bundled with Xcode
- lua-lsp** Lua
Binary: brew install lua-language-server
- php-lsp** PHP
Binary: npm i -g intelephense

What Claude gains

- Auto diagnostics** Errors after every edit
- Go to definition** Jump across files
- Find references** All usages of a symbol
- Hover types** Type info without compiling
- Call hierarchy** Trace callers and callees

Skill Frontmatter

- description** Auto-invocation trigger
- allowed-tools** Skip permission prompts
- model / effort** Override for this skill
- paths: [globs]** Path-specific activation
- context: fork** Run in subagent
- disable-model-invocation** Manual-only skill
- \$ARGUMENTS** User input placeholder
- `\${CLAUDE\_SKILL\_DIR}`** Skill's own directory
- `cmd`** Dynamic context injection

Agent Frontmatter

- name / description** Required
- model** Override model per agent
- effort** Override effort per agent
- maxTurns** Cap agentic turns
- tools / disallowedTools** Restrict tool access
- skills** Specific skills to expose
- memory** user / project / local
- background: true** Run as background task
- isolation: worktree** Run in isolated git worktree
- initialPrompt** Auto-submit first turn
- mcpServers** Agent-specific MCP servers
- hooks:** Per-agent hook decs

OFFICIAL PLUGINS

Marketplace: [claude-plugins-official](#) · Browse: [claude.com/plugins](#)

Source Control

- github** PRs, issues, repos, Actions
- gitlab** MRs, issues, pipelines, repos

Project Management

- atlassian** Jira tickets + Confluence docs
- asana** Tasks, projects via Asana MCP
- linear** Issues, cycles via Linear MCP
- notion** Pages, databases, search

Design

- figma** Files, components, variables, designs

Infrastructure & Backend

- vercel** Deploy, inspect Vercel projects
- firebase** Firestore, Auth, Storage, Hosting
- supabase** Postgres, Auth, Edge Functions

Communication & Monitoring

- slack** Post messages, read channels
- senry** Query errors, issues, replays

Development Workflows

- commit-commands** Git commit/push/PR workflows
- pr-review-toolkit** Specialised PR review agents
- code-review** Replaces deprecated /review
- agent-sdk-dev** Build with Claude Agent SDK
- plugin-dev** Create & debug your own plugins

Output Styles

- explanatory-output-style** Educational insights
- learning-output-style** Interactive learning mode

Quick Install

- /plugin install github@claude-plugins-official*
- /plugin install typescript-lsp@claude-plugins-official*
- /plugin install commit-commands@claude-plugins-official*

PLUGIN MANAGEMENT

CLI Plugin Commands

- claude plugin install <n>** Install from marketplace
- claude plugin uninstall <n>** Remove (–keep-data opt.)
- claude plugin enable <n>** Re-enable disabled plugin
- claude plugin disable <n>** Disable without uninstalling
- claude plugin update <n>** Update to latest version
- claude plugin list** List installed (–json)
- claude plugin list --available** Include marketplace catalog
- claude plugin tag** Create release git tag
- claude plugin validate** Check schemas

Slash Commands

- /plugin* Open plugin manager UI
- /plugin install <n>@<mk>* Install from session
- /plugin marketplace add <src>* Add from GitHub/URL/path
- /plugin marketplace list* List marketplaces
- /plugin marketplace update* Refresh catalog
- /plugin marketplace remove* Remove marketplace
- /reload-plugins* Hot-reload mid-session

Plugin Directory Structure

- my-plugin/*
- .claude-plugin/plugin.json** Manifest (optional)
- skills/<n>/SKILL.md** Slash command skills
- commands/<n>.md** Flat commands (older style)
- agents/<n>.md** Subagent definitions
- hooks/hooks.json** Event hooks
- .mcp.json** MCP server config
- .lsp.json** LSP server config
- monitors/monitors.json** Background monitors
- themes/<n>.json** Custom colour themes
- bin/** Executables → Bash PATH
- scripts/** Hook & utility scripts

Plugin Path Variables

- `\${CLAUDE\_PLUGIN\_ROOT}`** Install dir (changes on update)
- `\${CLAUDE\_PLUGIN\_DATA}`** Persistent data — survives updates
- `\${user\_config.KEY}`** User-supplied config value

Checkpointing (/rewind)

- /rewind** Rewind conv and/or code
- /checkpoint** Alias for /rewind
- /undo** Alias for /rewind
- Esc Esc** Rewind / summarize from point

Disable: `CLAUDE_CODE_DISABLE_FILE_CHECKPOINTING=1`

★ A+ TIPS

- /reload-plugins** after install = no restart
- --scope** project shares via git
- plugin validate** catches YAML errors
- Omit version in plugin.json → git-SHA
- `\${CLAUDE\_PLUGIN\_DATA}`** survives updates
- Press f in /plugin Installed = favourite
- Ctrl+O shows LSP diagnostics indicator

COLOUR KEY: Keyboard Slash cmd Workflow Skill/agent CLI/flag Config/env ★ New in 2026 Removed/deprecated

Use Case — Building an Excellent Obsidian Vault

Treat your vault as a repo — Claude Code does the rest

An Obsidian vault is just a folder of Markdown files — which makes it an **ideal Claude Code project**. Point Claude at your vault, add a **CLAUDE.md** at the root describing your conventions, codify repeated note operations as **skills in `.claude/skills/`**, and use **plan mode** before any bulk change. Version-control the vault with git so you can diff, branch, and roll back entire restructures. The rest of this page shows the setup, the templates, the bulk operations, and the custom skills that turn a messy notes folder into a durable knowledge system.

■ SETUP & STRUCTURE

Initial Setup (one-time)

cd ~/MyVault Enter your vault directory
git init && git add . Version control the vault
claude Start Claude Code in the vault
/init Have Claude draft CLAUDE.md
mkdir .claude/skills Create skills directory

PARA Structure

Vault/
■ 1-Projects/ active, deadline-driven
■ 2-Areas/ ongoing responsibilities
■ 3-Resources/ reference material
■ 4-Archive/ completed / inactive
■ Slipbox/ atomic permanent notes
■ Daily/ daily notes
■ Templates/ note templates
■ _attachments/ images, PDFs
■ CLAUDE.md vault conventions

Zettelkasten Structure (alt.)

Vault/
■ Inbox/ fleeting notes (processed weekly)
■ Literature/ source-based notes
■ Permanent/ atomic ideas (one-idea-per-note)
■ MOCs/ Maps of Content (index notes)
■ CLAUDE.md

Example CLAUDE.md (vault root)

Vault Conventions

Note types & folders

- Daily: `Daily/YYYY-MM-DD.md`
- Permanent: `Slipbox/` — atomic ideas
- Literature: `Literature/` — source notes
- MOC: `MOCs/` — topic indexes

Frontmatter (required on all notes)

type: [daily|permanent|lit|moc]
created: YYYY-MM-DD
tags: [topic1, topic2]
source: <url or citation>

Linking

- Use [[wikilinks]], not markdown links
- Every permanent note links to ≥1 other
- MOCs use bulleted [[links]]

Tag hierarchy

- #area/work, #area/personal
- #topic/ml, #topic/writing
- #status/draft, #status/reviewed

■ TEMPLATES & CONVENTIONS

*{{date:...}} syntax requires the Templater plugin;
core Templates plugin uses {{date}} without format.*

Daily Note Template

type: daily
created: {{date:YYYY-MM-DD}}
tags: [daily]

{{date:dddd, MMMM D YYYY}}

■ Focus

-

■ Notes

-

■ Links

- [[Yesterday]] ← [[Today]] → [[Tomorrow]]

■ Ideas captured

-

Permanent Note Template

type: permanent
created: {{date:YYYY-MM-DD}}
tags: []
links_to: []

{{title}}

<!-- ONE idea, in your own words -->

Why it matters

Related

- [...]

Source

- [[Literature/...]]

Literature Note Template

type: lit
author:
year:
url:
tags: []

{{title}}

Summary (one paragraph)

Key ideas

-

Quotes

>

Permanent notes extracted

- [[Slipbox/...]]

■ BULK OPS & CUSTOM SKILLS

Useful Prompts (use /plan first!)

- Add missing frontmatter to all notes, infer type from folder
- Find orphan notes (zero backlinks) in /Slipbox and list them
- Generate MOCs/Machine-Learning.md linking every #topic/ml note
- Find duplicate or near-duplicate notes; suggest merges in a report
- Extract permanent notes from last month's daily notes into /Slipbox
- Rename all files to kebab-case, update every [[wikilink]] referencing them
- Convert markdown links to [[wikilinks]] where the target is in-vault
- Audit tag usage; propose a consolidated hierarchy
- Find notes with broken [[wikilinks]] and list targets
- Migrate this Notion export to vault using our conventions

Custom Skills to Build

*Suggested patterns for .claude/skills/<n>/SKILL.md
— write each skill yourself; names are illustrative*

/daily Open today's daily note; create if absent

/weekly-review Review week: extract insights → /Slipbox

/moc <topic> Build Map of Content for #topic/*

/orphans List notes with no in/out links

/frontmatter-fix Add/repair frontmatter across vault

/link-suggest Suggest links for current note

/atomicize Split multi-idea note into atomic notes

/lit <url> Fetch article → draft literature note

/tag-audit Report tag usage + hierarchy warnings

/inbox-process Triage Inbox/ notes by type

Safe-Ops Workflow

1. git commit before any bulk change
2. Enter /plan mode — review the approach
3. Preview with /diff after each turn
4. Major restructures → use --worktree
5. Commit working state; iterate in small steps
6. Ask /btw for meta questions mid-task

★ EXPERT TIPS

- Keep CLAUDE.md short & imperative
- Store templates in /Templates — Claude reads them like any other note
- ``.obsidian/workspace*.json`` in .gitignore
- Use acceptEdits mode for trusted ops, default mode for destructive ones
- Daily note links should form a chain
- Run /tag-audit monthly, not per-session